Loyalty Platform

Documentação Final Java (Sprint 4)

API RESTful em Spring Boot integrada ao front-end em React

Equipe: HalfTech
Turma: 1TDSPW

FIAP Challenge 2026 · Domain Driven Design using Java

Integrante	RM	Turma
Andrei Oliveira de Sousa	573373	1TDSPW
Felipe Coelho da Fonseca	572133	1TDSPW
Henrique Soares Pereira	569898	1TDSPW
Luiz Felipe Lourenço da Silva	572576	1TDSPW

Novembro de 2026

1. Objetivo e escopo

A Loyalty Platform é uma plataforma B2B de gamificação: empresas clientes criam campanhas com missões, e os usuários delas acumulam pontos, níveis, ranking e sequência de dias. O objetivo é ajudar essas empresas a reter os próprios usuários sem construir essa estrutura do zero.

Nesta sprint, o escopo foi a API RESTful em Spring Boot, ligada ao banco Oracle e integrada ao front-end em React. Ela atende o painel do administrador da Loyalty (gestão das empresas clientes) e deixa prontos os endpoints do painel da empresa cliente (campanhas, missões, ranking e métricas), cujas telas ainda não foram feitas.

2. Funcionalidades

- **Login único com dois perfis:** ADMIN (equipe Loyalty) e EMPRESA (empresa cliente), com senha guardada em BCrypt e token JWT válido por 8 horas.
- **Empresas clientes:** cadastro, consulta, edição, inativação e reativação. Inativar encerra as campanhas ativas e tira o acesso da empresa; empresa não é excluída.
- **Identidade visual por empresa:** cor principal, tipografia e formato dos botões do app (white label).
- **Campanhas:** nascem inativas, só ativam com pelo menos uma missão ativa, e campanha encerrada não volta. Só é possível excluir rascunho sem participantes.
- **Missões:** meta e pontos inteiros positivos, e exclusão lógica que preserva o histórico.
- **Ranking e métricas de engajamento** por empresa e por campanha, a partir das consultas da entrega de BD.
- **Validação de todos os dados de entrada**, com mensagens em português por campo, tratamento centralizado de exceções e CORS liberado para o front.

3. Tabela de endpoints

A API tem 21 endpoints. Fora `GET /` e `POST /auth/login`, todos exigem o cabeçalho `Authorization: Bearer <token>`. "EMPRESA dona" significa que o perfil EMPRESA só acessa os dados da própria empresa.

Verbo	URI	O que faz	Quem pode	Sucesso	Erros
GET	`/`	Confere se a API está no ar	Pública	200	—
POST	`/auth/login`	Login: devolve o token e os dados do usuário	Pública	200	400, 401, 403
GET	`/empresas`	Lista todas as empresas	ADMIN	200	401, 403
GET	`/empresas/{id}`	Detalhe de uma empresa	ADMIN ou EMPRESA dona	200	400, 401, 403, 404
POST	`/empresas`	Cadastra empresa (nasce ATIVA)	ADMIN	201	400, 401, 403, 409
PUT	`/empresas/{id}`	Atualiza dados cadastrais e identidade visual	ADMIN	200	400, 401, 403, 404, 409
PATCH	`/empresas/{id}/status`	Inativa (encerra as campanhas ativas) ou reativa	ADMIN	200	400, 401, 403, 404, 409
GET	`/empresas/{idEmpresa}/campanhas`	Lista as campanhas com os totais de missões	ADMIN ou EMPRESA dona	200	400, 401, 403, 404
POST	`/empresas/{idEmpresa}/campanhas`	Cria campanha (nasce INATIVA)	ADMIN ou EMPRESA dona	201	400, 401, 403, 404, 409
GET	`/campanhas/{id}`	Detalhe da campanha com as missões	ADMIN ou EMPRESA dona	200	400, 401, 403, 404
PUT	`/campanhas/{id}`	Edita nome, descrição e datas	ADMIN ou EMPRESA dona	200	400, 401, 403, 404, 409
PATCH	`/campanhas/{id}/status`	Muda para ATIVA, INATIVA ou ENCERRADA	ADMIN ou EMPRESA dona	200	400, 401, 403, 404, 409
DELETE	`/campanhas/{id}`	Exclui rascunho inativo sem participantes	ADMIN ou EMPRESA dona	204	400, 401, 403, 404, 409
GET	`/campanhas/{idCampanha}/missoes`	Lista as missões da campanha	ADMIN ou EMPRESA dona	200	400, 401, 403, 404
POST	`/campanhas/{idCampanha}/missoes`	Cria missão (nasce ATIVA)	ADMIN ou EMPRESA dona	201	400, 401, 403, 404, 409
GET	`/missoes/{id}`	Detalhe da missão	ADMIN ou EMPRESA dona	200	400, 401, 403, 404
PUT	`/missoes/{id}`	Edita nome, descrição, meta e pontos	ADMIN ou EMPRESA dona	200	400, 401, 403, 404, 409
PATCH	`/missoes/{id}/status`	Muda para ATIVA ou INATIVA	ADMIN ou EMPRESA dona	200	400, 401, 403, 404, 409
DELETE	`/missoes/{id}`	Exclusão lógica (status EXCLUIDA)	ADMIN ou EMPRESA dona	204	400, 401, 403, 404, 409
GET	`/empresas/{idEmpresa}/ranking`	Ranking de usuários da empresa	ADMIN ou EMPRESA dona	200	400, 401, 403, 404
GET	`/empresas/{idEmpresa}/metricas`	Engajamento geral e por campanha	ADMIN ou EMPRESA dona	200	400, 401, 403, 404

Todos os erros vêm no formato `{ status, mensagem, campos }`; `campos` só aparece quando há erro em um campo específico. Qualquer rota pode responder 500 se o banco estiver fora do ar.

Código	Quando acontece
400	Dados inválidos, com a mensagem de cada campo em `campos`, ou id não numérico na URL
401	E-mail ou senha incorretos, ou token ausente, inválido ou expirado
403	Perfil sem permissão, dados de outra empresa, ou login de empresa inativa
404	Registro não existe (inclusive missão excluída) ou rota inexistente

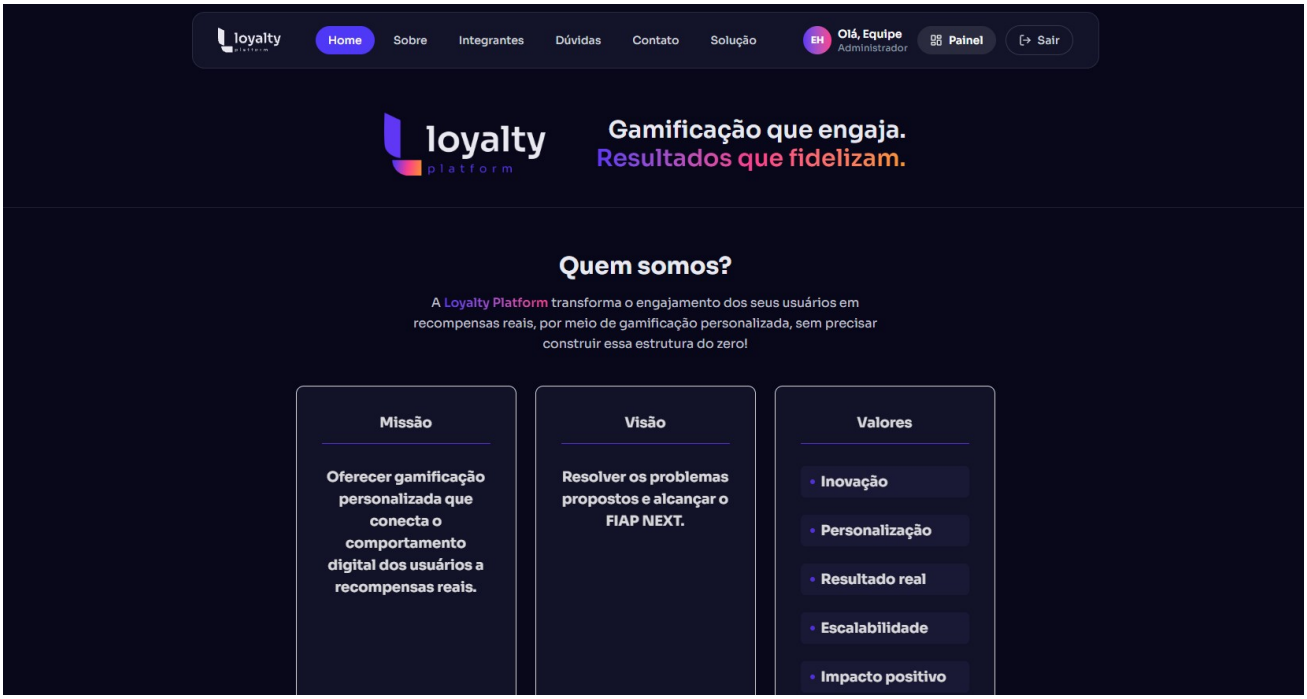
Código	Quando acontece
405	Verbo não suportado na rota, como `DELETE /empresas/{id}` (empresa é inativada, não excluída)
409	Regra de negócio: CNPJ ou e-mail duplicado, status já aplicado, campanha encerrada, ativar campanha sem missão ativa
415	Corpo enviado sem `Content-Type: application/json`
500	Erro inesperado ou banco fora do ar

4. Protótipo: prints das telas

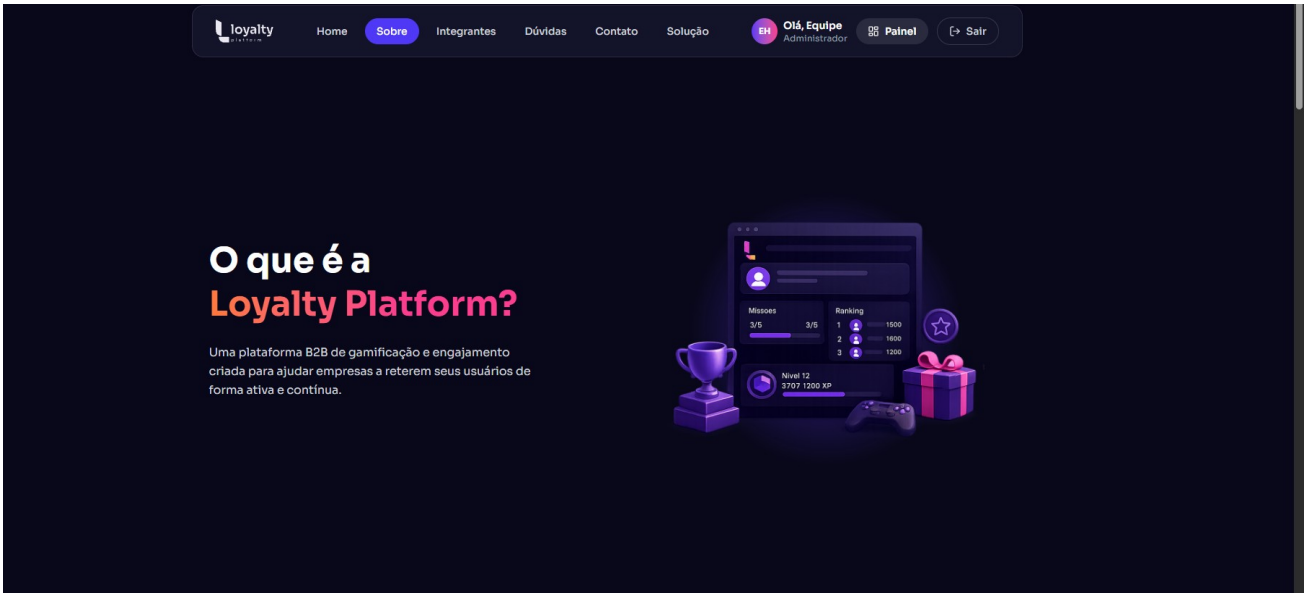
Os prints abaixo foram tirados com o front ligado à API real, sem o modo de API simulada.

4.1 Home e páginas institucionais

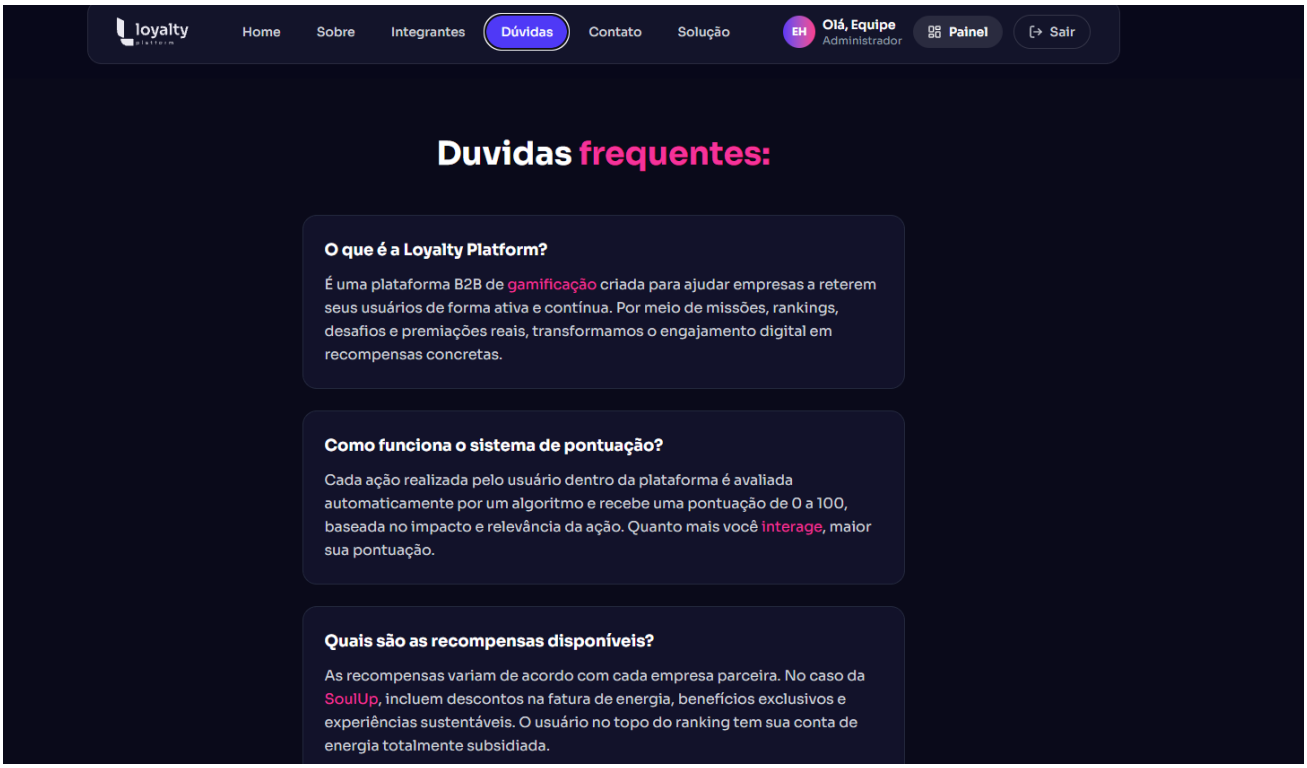
Rotas `/`, `/sobre`, `/solucao`, `/integrantes`, `/duvidas` e `/contato`: o site institucional da solução.



Home



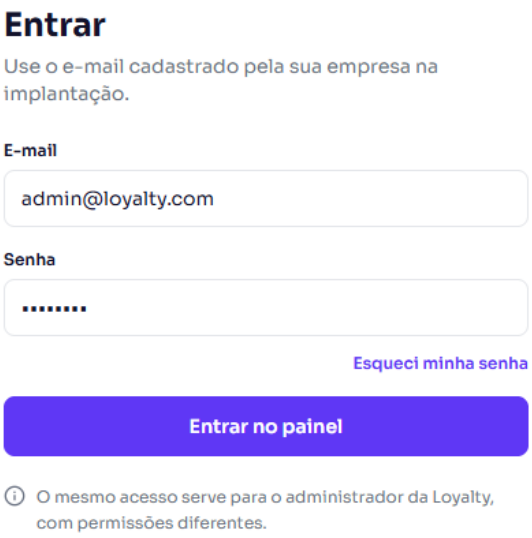
Sobre



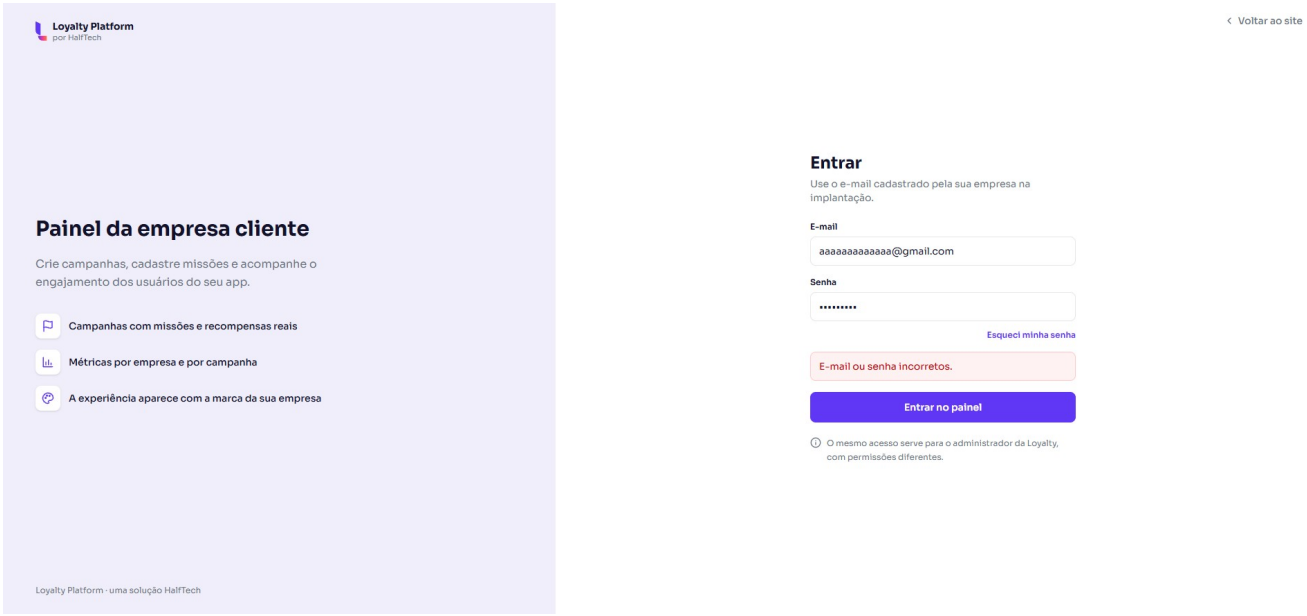
Dúvidas frequentes

4.2 Login

Rota `/login`: login único para os dois perfis. O segundo print mostra o erro "E-mail ou senha incorretos." devolvido pela API (401).



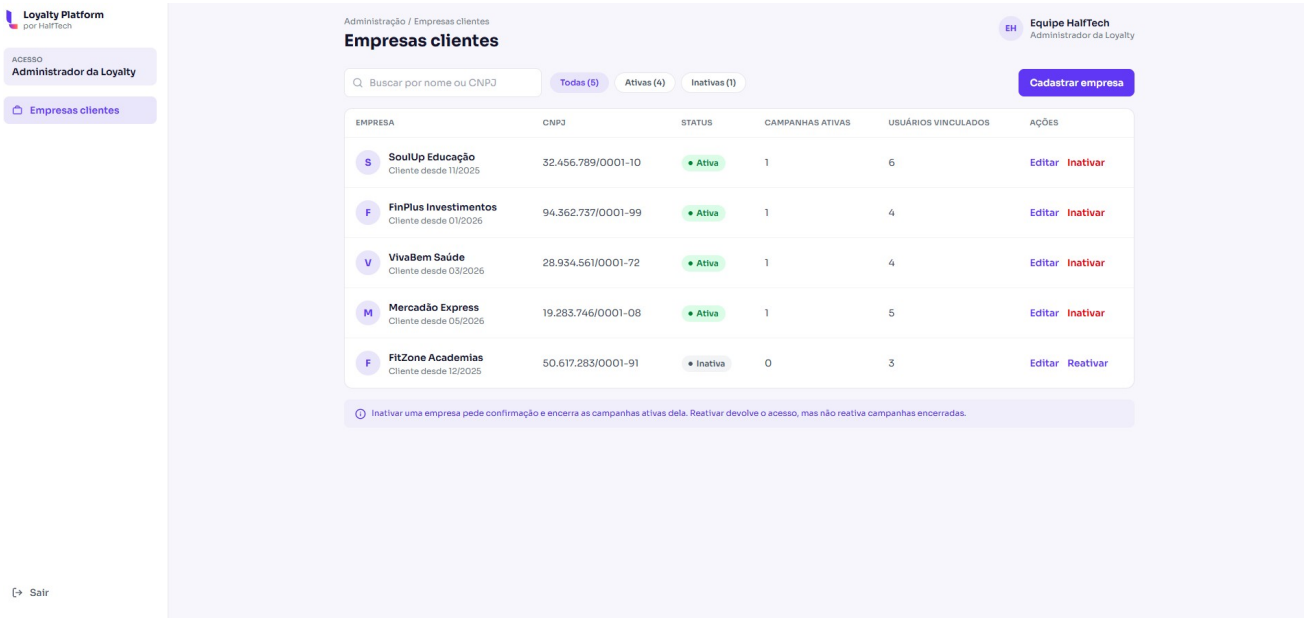
Login com a conta de administrador



Login recusado pela API

4.3 Empresas clientes

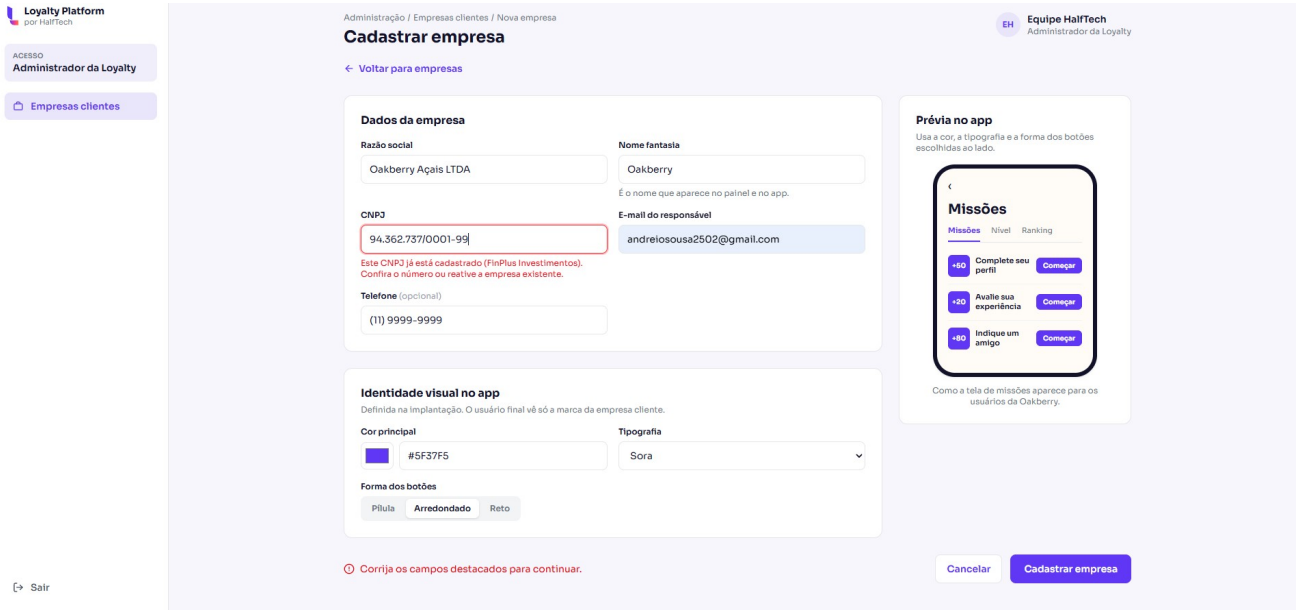
Rota `/admin/empresas`: lista vinda de `GET /empresas`, com busca por nome ou CNPJ e filtro por status.



Lista de empresas clientes vinda da API

4.4 Cadastrar empresa

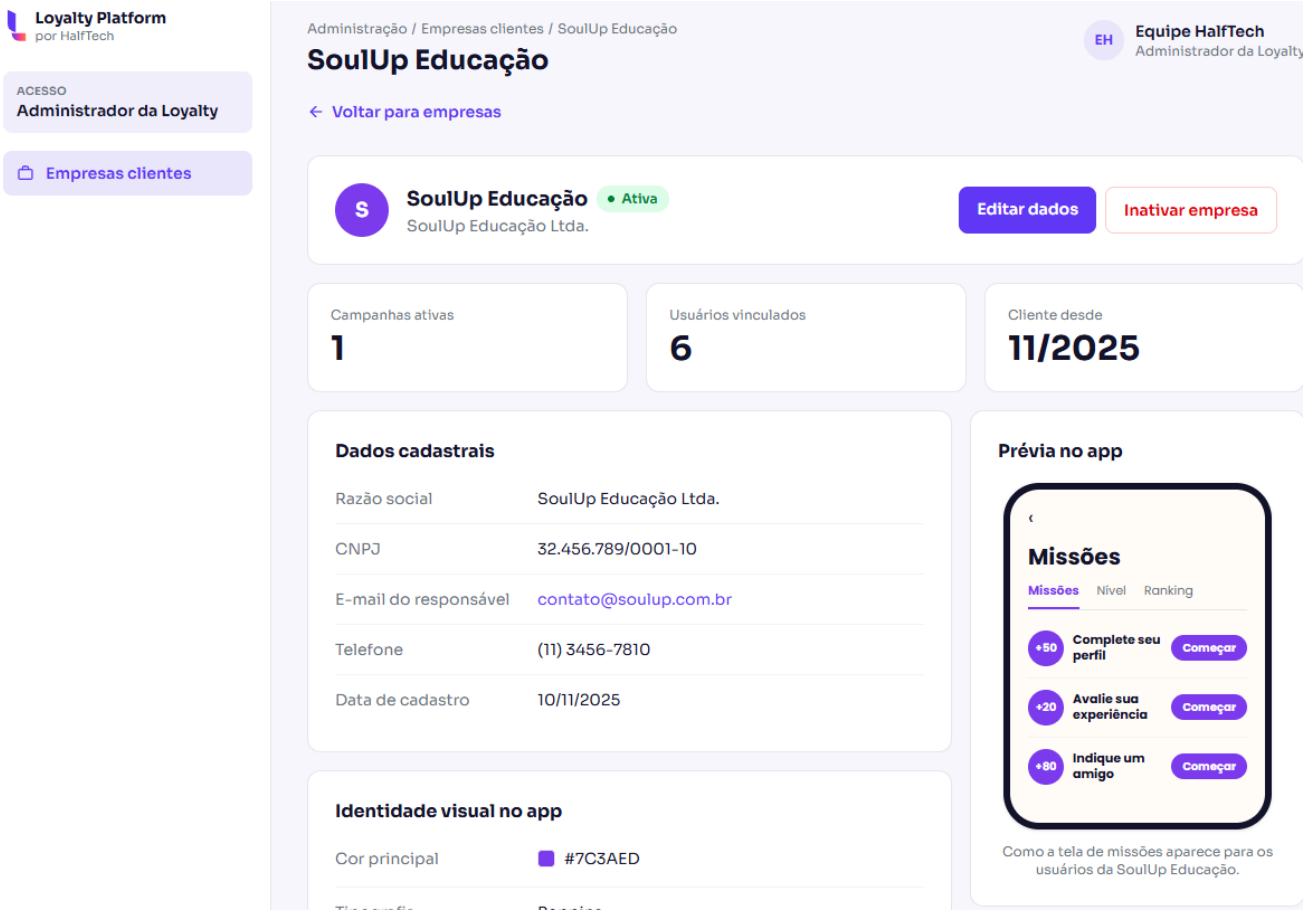
Rota `/admin/empresas/nova`: validação dos campos e aviso de CNPJ já cadastrado (409 da API).



Cadastro recusado pela API por CNPJ já cadastrado

4.5 Detalhe da empresa

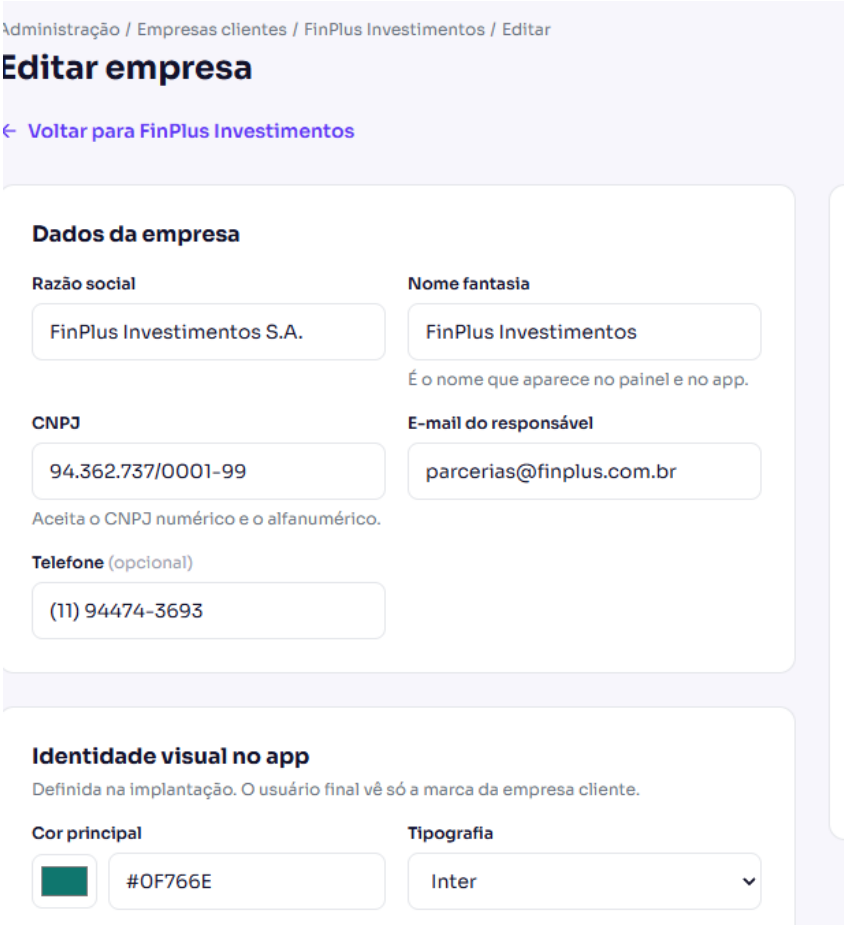
Rota `/admin/empresas/:id`: dados, identidade visual e os totais calculados pela API (campanhas ativas e usuários vinculados).



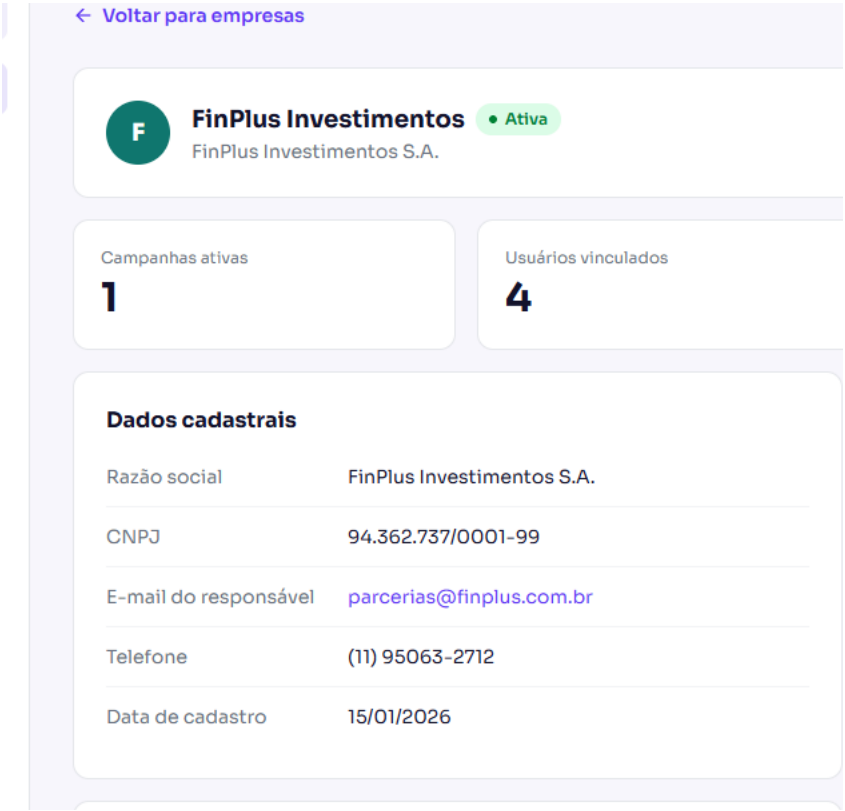
Detalhe da SoulUp Educação

4.6 Editar empresa

Rota `/admin/empresas/:id/editar`: edição com a prévia da identidade visual no app. Com o CNPJ validado só pelo formato, as empresas da carga inicial podem ser editadas e salvas pela tela.



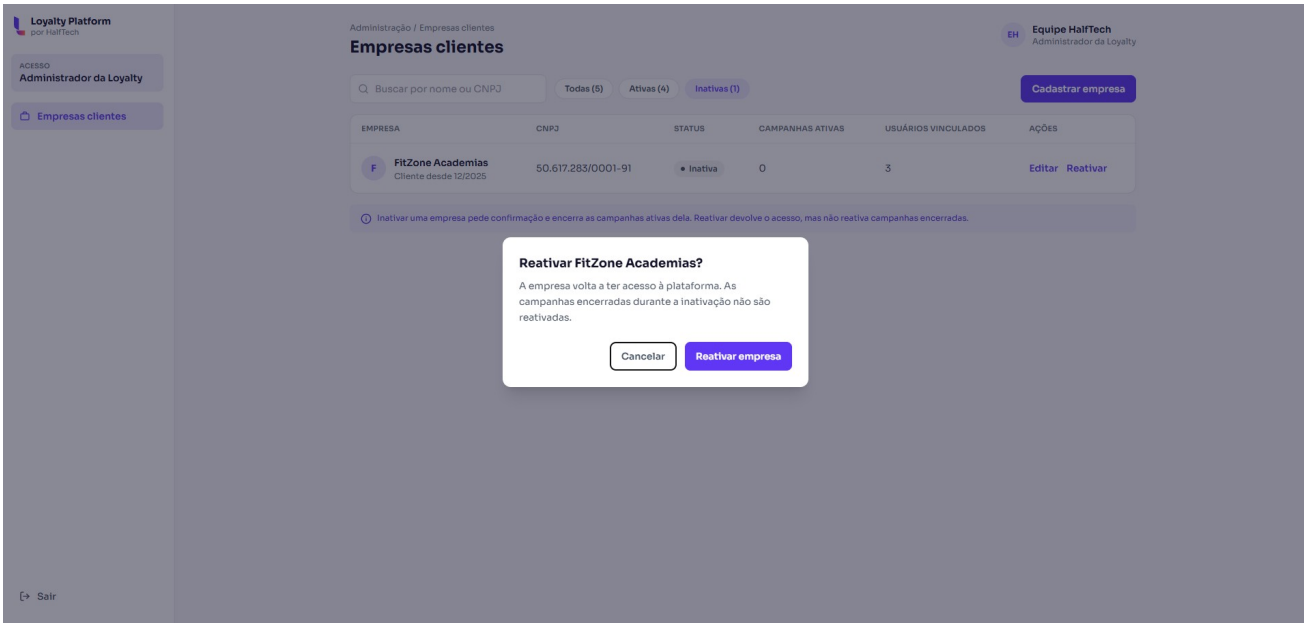
Formulário de edição da FinPlus Investimentos



Detalhe da FinPlus Investimentos

4.7 Confirmação de inativar e reativar

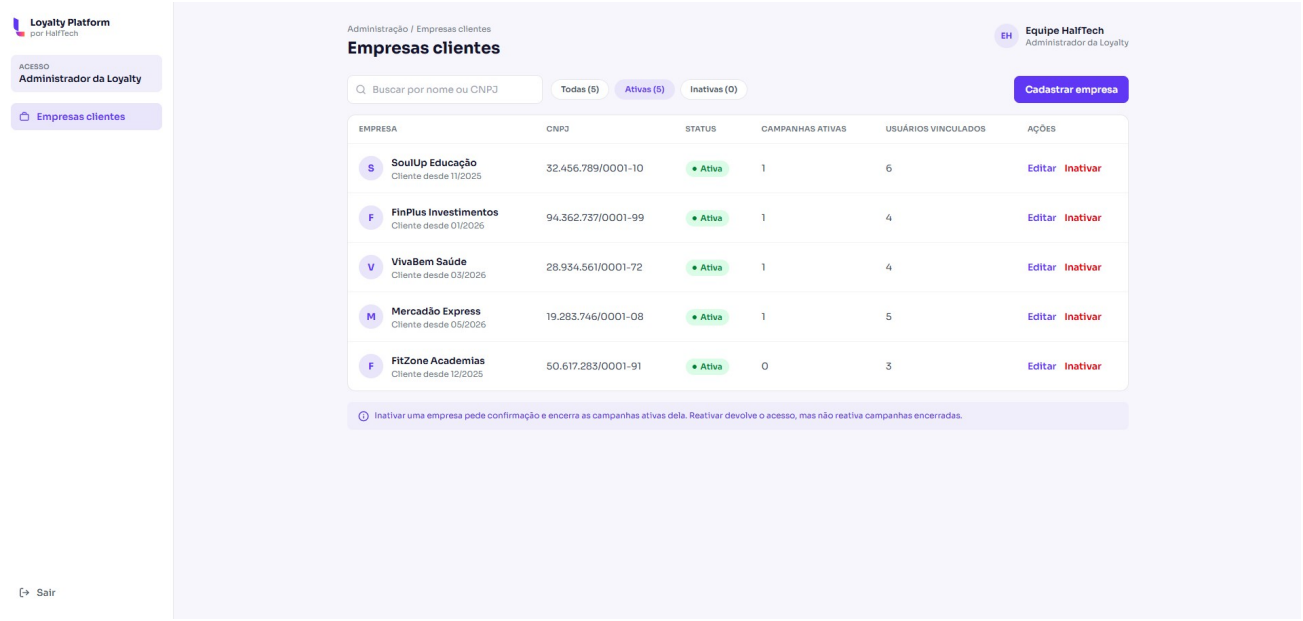
Modal em `/admin/empresas`: as duas ações pedem confirmação. Inativar avisa que as campanhas ativas serão encerradas; reativar avisa que as campanhas encerradas durante a inativação não voltam.



Confirmação para reativar a FitZone Academias

4.8 Empresa inativada e reativada

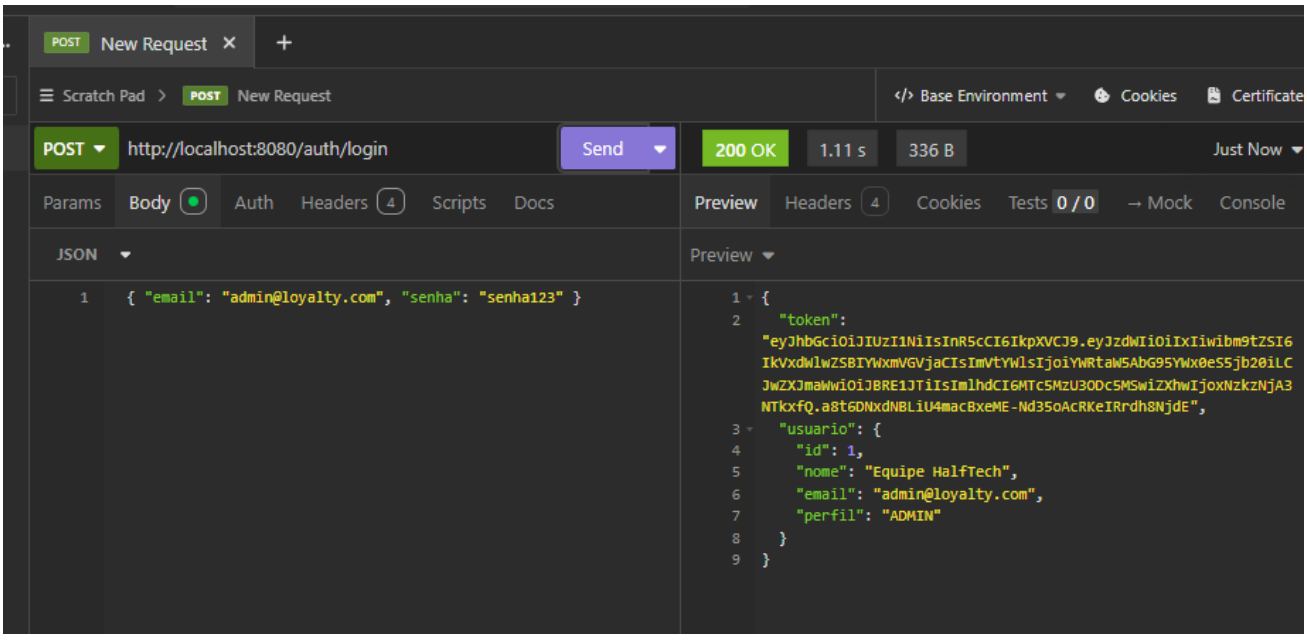
Rota `/admin/empresas`: inativa, a FitZone aparece com status "Inativa", 0 campanhas ativas e o botão "Reativar" (print da seção 4.3). Depois de reativada, volta para "Ativa", mas continua com 0 campanhas ativas, porque reativar não reativa campanhas encerradas.



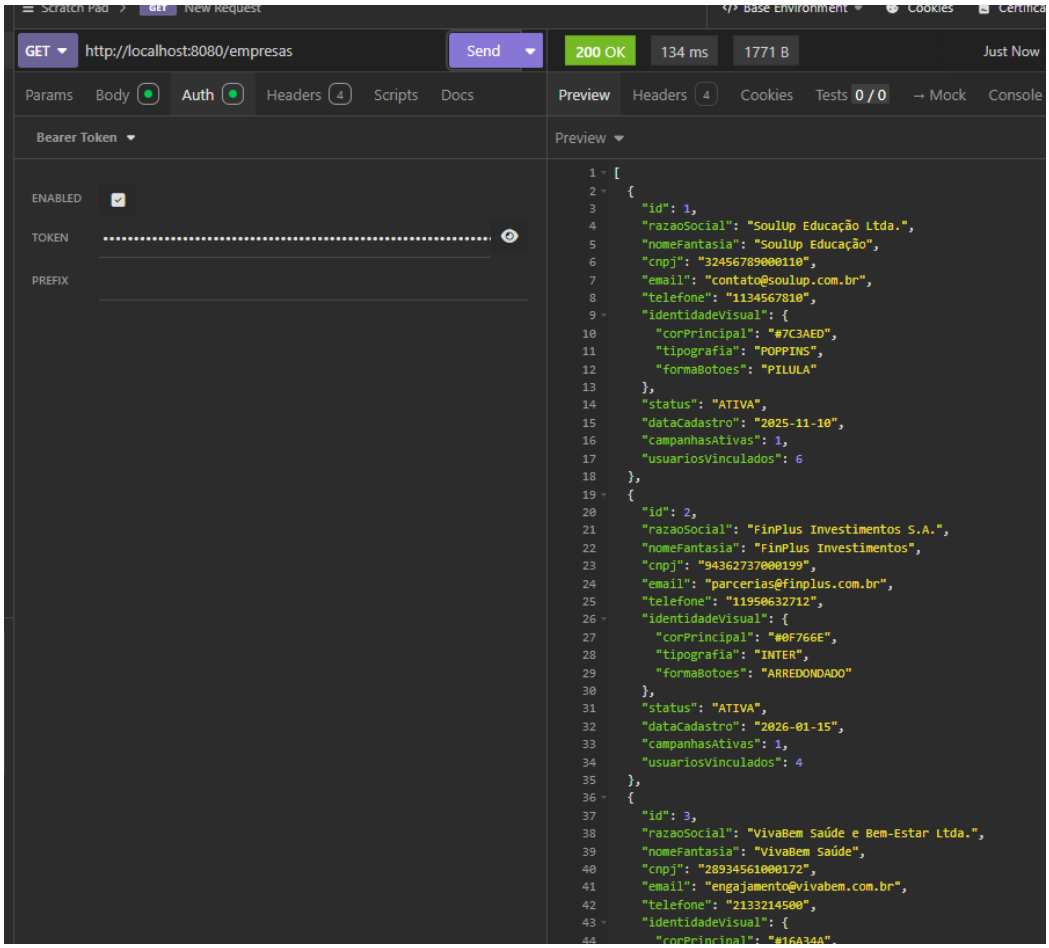
Lista depois de reativar a FitZone

4.9 Insomnia (evidência da API)

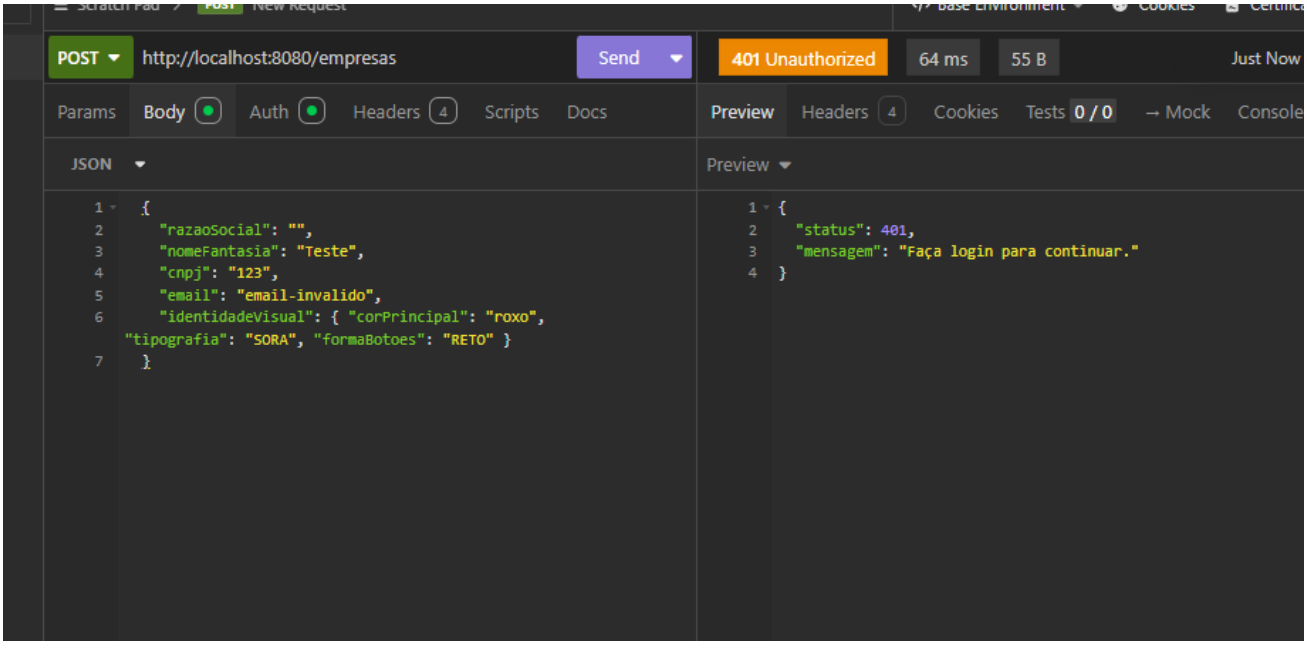
Chamadas feitas direto na API, sem o front: login (200 com o token), `GET /empresas` com o token (200) e `POST /empresas` sem o token (401, barrado pelo `AutenticacaoFilter`).



POST /auth/login respondendo 200 com o token



GET /empresas com o token respondendo 200



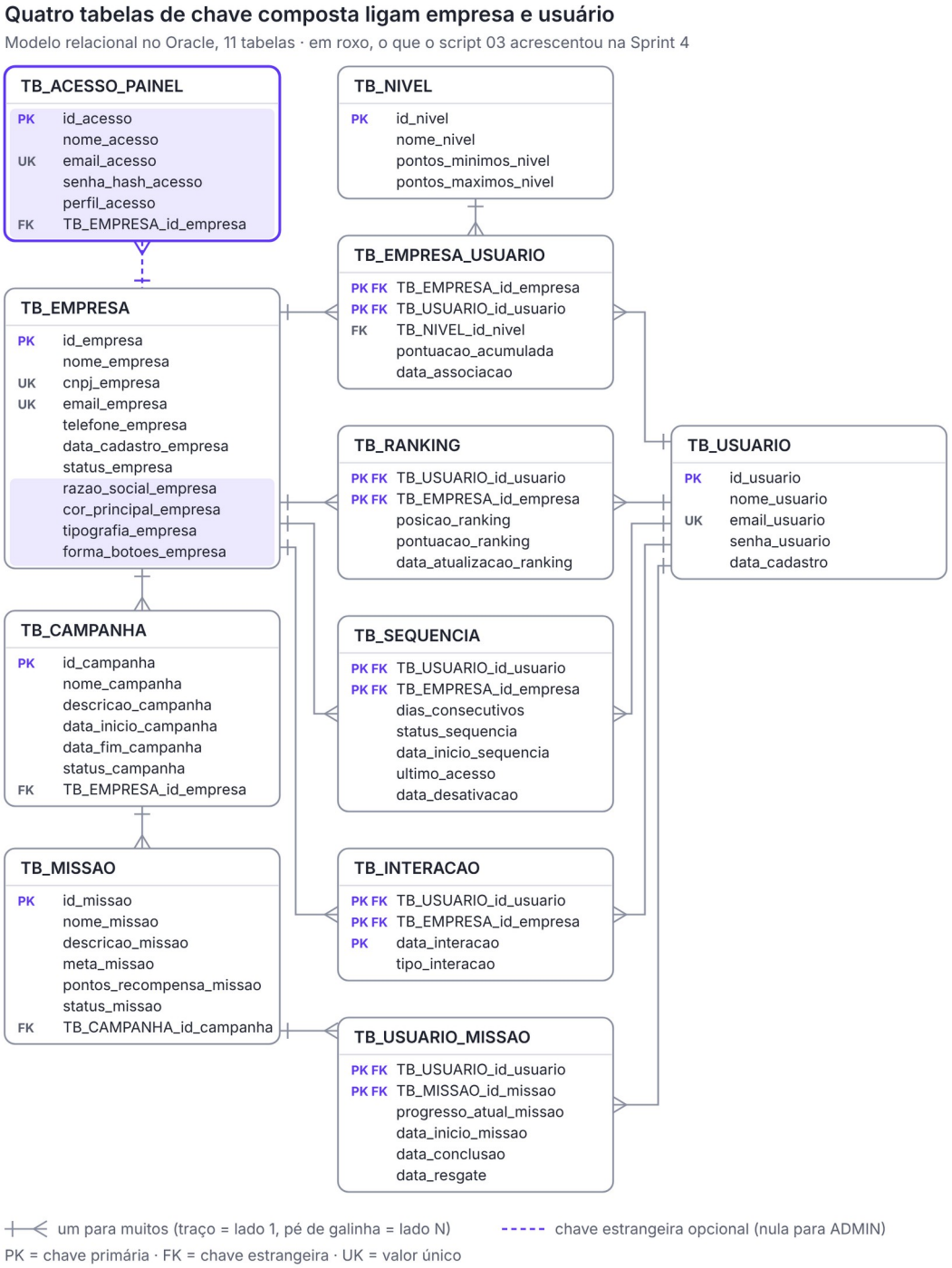
POST /empresas sem o token respondendo 401

Os demais endpoints seguem o mesmo padrão de token e de formato de erro, conforme a tabela de endpoints.

5. Modelo de entidade-relacionamento (MER)

O banco tem 11 tabelas: as 10 da entrega de BD (script `00_script_ddl_sprint3.sql`) e a `TB_ACESSO_PAINEL`, criada pelo script `03_alteracoes_api_java.sql`, que também acrescentou quatro colunas obrigatórias em `TB_EMPRESA`: razão social, cor principal, tipografia e formato dos botões.

Pontos, nível, ranking, sequência e interações ficam no vínculo entre empresa e usuário (chave composta), porque o mesmo usuário pode participar de mais de uma empresa cliente. `TB_ACESSO_PAINEL` guarda quem entra no painel web: o perfil ADMIN não tem empresa, e o perfil EMPRESA aponta para a empresa dele.



6. Diagrama de classes

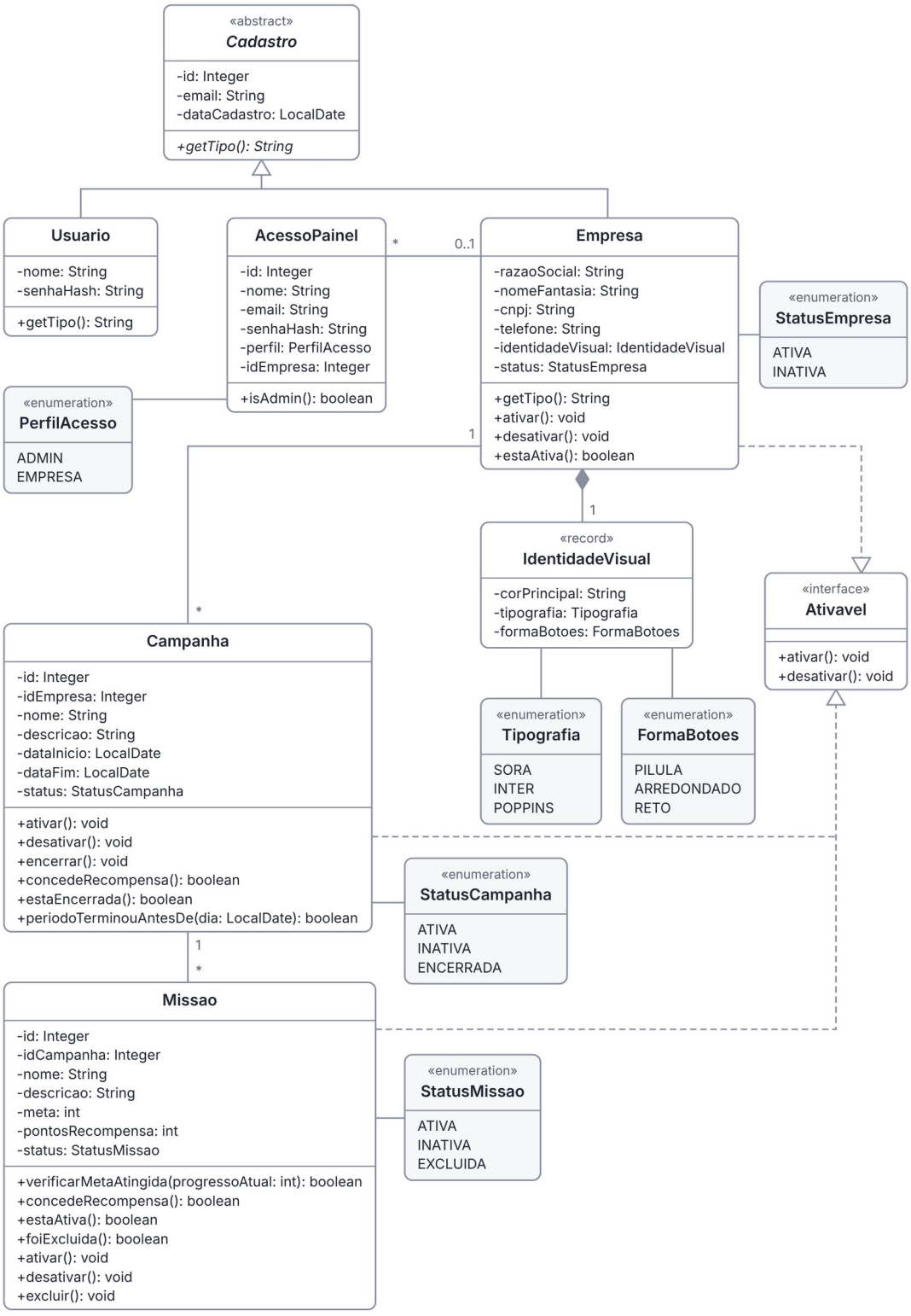
O diagrama segue o código entregue e está dividido em três partes para continuar legível: as duas primeiras mostram as classes do pacote `model`, com atributos, métodos e relações; a terceira mostra as camadas da API. Getters, setters e construtores foram omitidos.

`Empresa` e `Usuario` herdam da classe abstrata `Cadastro`; `Empresa`, `Campanha`, `Missao` e `Sequencia` implementam a interface `Ativavel`; `IdentidadeVisual` é um record composto dentro de `Empresa` (objeto de valor, sem id próprio).

Cada requisição passa pelo `CorsFilter` e pelo `AutenticacaoFilter`, chega ao Resource, que chama o BO. O BO aplica as regras de negócio, abre a conexão (e a transação) pela `ConnectionFactory` e passa essa conexão aos DAOs, que executam o SQL no Oracle.

Diagrama de classes 1 de 2: empresas, acesso ao painel, campanhas e missões

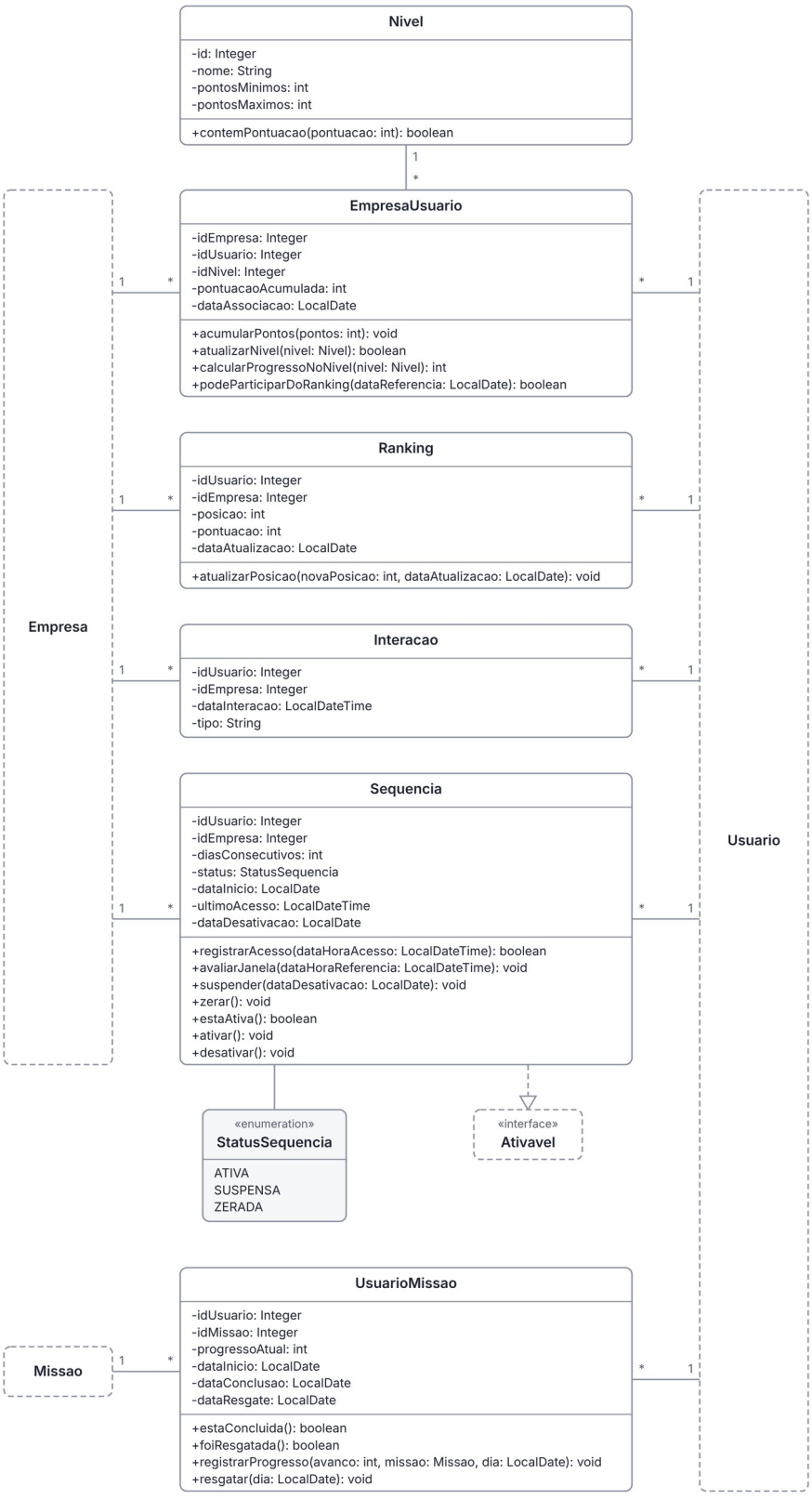
Pacote br.com.fiap.loyalty.model · Empresa e Usuario herdam de Cadastro; Empresa, Campanha e Missao implementam Ativavel



Triângulo vazado em linha cheia = herança · em linha tracejada = implementa a interface
Losango cheio = composição · linha simples = associação, com a multiplicidade nas pontas
Getters, setters e construtores omitidos. No código, as associações usam o id (ex.: Campanha.idEmpresa), como as chaves estrangeiras do banco.

Diagrama de classes 2 de 2: pontuação, nível, ranking e sequência

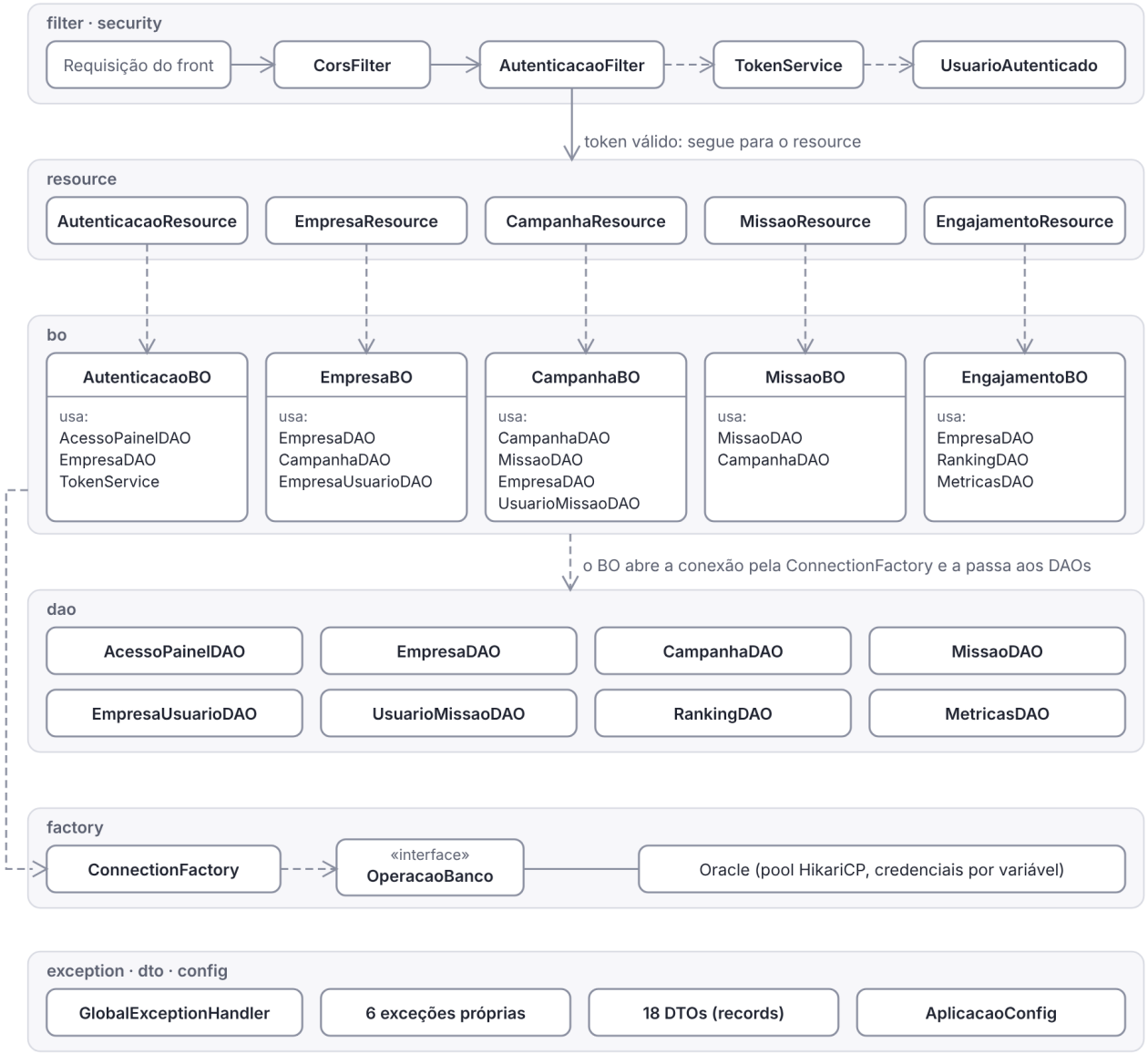
Pacote br.com.fiap.loyalty.model · o engajamento de cada usuário é guardado por empresa



As quatro classes do meio ligam um Usuario a uma Empresa (chave composta no banco).
Caixa tracejada = classe detalhada no diagrama 1 · triângulo vazado em linha tracejada = implementa a interface
Linha simples = associação, com a multiplicidade nas pontas · getters, setters e construtores omitidos

Camadas da API: Resource → BO → DAO → ConnectionFactory

Pacotes de br.com.fiap.loyalty · cada requisição passa pelos filtros e desce as camadas até o Oracle



Setas tracejadas = dependência (quem chama quem). Toda exceção, de qualquer camada, vira { status, mensagem, campos } no GlobalExceptionHandler. RaizResource (GET /, health check) responde direto, sem BO. Métodos omitidos neste diagrama.

7. Procedimentos para rodar a aplicação

Ferramentas: IntelliJ IDEA (Community ou Ultimate), SQL Developer para o banco e Postman ou Insomnia para testar. O Maven já vem dentro do IntelliJ.

Versões: JDK 21 ou mais novo (o projeto compila para Java 21), Spring Boot 4.1.1 e Oracle Database 19c (driver ojdbc17).

1. **Clonar e importar.** Rode `git clone https://github.com/Challenge-LoyaltyPlatform/Java-LoyaltyPlatform4.git` e, no IntelliJ, use File → Open na pasta clonada. Ele reconhece o `pom.xml` e baixa as dependências; espere a barra de progresso terminar.
2. **Configurar o JDK.** Se o IntelliJ pedir um SDK, vá em File → Project Structure → Project → SDK → Download JDK e escolha a versão 21 (Eclipse Temurin).
3. **Criar o `.env`.** Copie o arquivo `.env.example` da raiz para `.env`, na mesma pasta, e preencha `DB_USERNAME` (o RM, ex.: `rm572133`) e `DB_PASSWORD`. Se definir `JWT_SECRET`, ele precisa ter 32 caracteres ou mais. O `.env` fica fora do Git.
4. **Preparar o banco.** No SQL Developer, conecte em `oracle.fiap.com.br`, porta `1521`, SID `ORCL`. Abra cada script da pasta `database/` e rode com F5 (Executar Script), nesta ordem:
    a. `00_script_ddl_sprint3.sql`: apaga e recria as tabelas do schema.
    b. `01_carga_inicial.sql`: carga de dados da entrega de BD.
    c. `03_alteracoes_api_java.sql`: colunas novas de `TB_EMPRESA` e contas de login do painel.
5. **Subir a API.** Abra `src/main/java/br/com/fiap/loyalty/LoyaltyPlatformApplication.java` e clique no ▶️ verde ao lado do `main`. Em `http://localhost:8080/` deve aparecer `{"status":"UP", ...}`.
6. **Testar.** No Postman ou Insomnia, envie `POST http://localhost:8080/auth/login` com `{"email": "admin@loyalty.com", "senha": "..."}` (a senha das contas de teste está no comentário do script 03). Copie o `token` e mande-o nas outras chamadas no cabeçalho `Authorization: Bearer <token>`.
7. **Ligar o front.** No repositório do front, pasta `Loyalty-Platform`, crie o `.env.local` com `VITE_API_URL=http://localhost:8080`, rode `npm install` e `npm run dev` e abra `http://localhost:5173`.

8. Links

Item	Link
Repositório da API (GitHub)	https://github.com/Challenge-LoyaltyPlatform/Java-LoyaltyPlatform4
Repositório do front-end (GitHub)	https://github.com/Challenge-LoyaltyPlatform/LoyaltyPlatform-Front-React
API publicada (Render)	https://loyalty-platform-api-snkp.onrender.com
Front-end publicado (Vercel)	https://loyalty-platform-front-react.vercel.app

No plano gratuito do Render, a API dorme depois de 15 minutos sem uso; a primeira chamada depois disso leva cerca de 1 minuto.